Basketball Action Recognition

Mateo Tomeho
#1009985296, mateo.tomeho@mail.utoronto.ca
Link to Github: https://github.com/mateotomeho/ActionRecognitionBasketball

1. Project Description - Basketball Action Recognition

This document presents my process in developing a deep learning application for action recognition in basketball. Motivated by the rapid growth of sports analytics and my personal background as a basketball player, the goal is to build a deep learning model that can classify key basketball actions such as shooting, dribbling, passing, and defence. Deep learning is an appropriate tool for this project because basketball action recognition relies on identifying complex spatial and temporal patterns within the video frames. This is something that machine learning methods can not capture as easily as Convolutional Neural Networks, which excel at learning & processing visual and temporal data. Therefore, my model would be able to process video frames to classify basketball actions from players effectively. This will be useful and interesting because this deep neural network could be applied to automate highlight generation, analyze a player's performance and contribute to real-time broadcasting by giving some assistance with the actions occurring during a basketball game.

2. Illustration

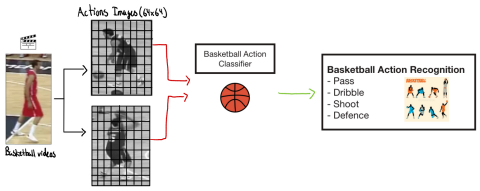

Figure 1: Overall Illustration of the Basketball Action Recognition Project

3. Background & Related Work

Using deep learning, many projects have been developed to recognize sports actions, including those in basketball, demonstrating the feasibility of implementing such a system in real-world contexts. The following is a brief overview of five related works on basketball action recognition.

This project [1] investigates the recognition of basketball technique actions using a three-dimensional (3D) Convolutional Neural Networks (CNNs) to automate the identification of various actions in basketball. Using 3D convolutions and Long Short-Term Memory (LSTM) networks, the model was able to get spatiotemporal relationships and temporal information of actions. The training was conducted on three public datasets: NTURGB+D, Basketball-Action-Dataset, and B3D Dataset. The proposed method achieved a 15.1% higher accuracy than the frame difference-based method and a 12.4% improvement over the optical flow method, confirming the effectiveness of the 3D CNN approach for basketball action recognition.

In this paper [2], a model was built to recognize complex actions for basketball players using the NPU RGB+D dataset. The authors created a new feature-enhanced skeleton-based method called LSTM-DGCN for the recognition that is based on a deep graph convolutional network (DGCN) and long short-term memory (LSTM) methods. It achieved up to 83.7% accuracy on their dataset, outperforming baseline methods and approaching state-of-the-art performance.

In this study [3], 3 modules were built to recognize basketball referee gestures (BRGR-Net). Using an Alterable Kernel Convolution (AKConv), a Coordinate Attention (CoordAtt) mechanism and a

Focal Loss Function, the project was able to address the dynamic gesture, background interference and imbalances of the referee. Using a novel basketball referee gesture dataset, BRGR-Net achieved 91.2% precision in hand gesture detection.

This project [4] explores the usage of a deep neural network based on a dynamic residual attention mechanism for basketball action Recognition. The model improves upon the traditional Convolutional 3D (C3D) network by better capturing key frames and spatiotemporal features in video sequences. It will achieve an average accuracy of more than 97% for posture recognition.

In this study [5], the authors address the challenges in basketball technical action recognition by proposing a 3D convolutional neural network framework. It operates on 2 different resolution image inputs using a dataset of 1800 basketball technical action videos. The model was able to achieve an accuracy of 96.9% proving its efficiency.

4. Data Processing

For this project, the data was collected from the Space Jam Dataset [6][7], which contains labelled videos of single player basketball actions, including shooting, dribbling, defence and passing. The dataset found is organized into three main components: video files of basketball actions, a label dictionary mapping each action and its corresponding numeral label and an annotations file that associates each video with its respective action label in a list.

To prepare the dataset for training, I performed the following data cleaning and formatting steps: I evaluated the class balance by checking the number of videos per action category and ensuring there were no unlabeled or missing videos (See Appendix - Figure 5). To simplify the training, I organized the dataset into subfolders named for each action: block, pass, run, dribble, shoot, ball in hand, defense, pick, no_action, and walk, and moved each video accordingly, removing the empty "discard" category. Finally, I split each video into 16 frames of 128×128 pixels, creating subfolders for individual videos and their frames, preparing the dataset for both the baseline and primary models. My entire dataset contains 593,456 images for all the videos.

The dataset was divided into training (80%), validation (10%) and test (10%) sets to ensure proper evaluation and generalization of the model. I focused on four action categories: passing, dribbling, shooting and defence to reduce the computational complexity of the training. For each category, I collected up to 500 video samples, each consisting of 16 frames, to ensure a balanced dataset and ensure the feasibility of the project. To enhance model generalization and reduce computational complexity, I applied grayscale conversion to all frames. The grayscale transformation helps reduce input size and memory usage without significantly compromising the information content. Also, it helps the model to focus on motion and shape-based features to recognize the basketball actions.

5. Architecture

As shown in Figure 2, the final neural network model architecture is the R(2+1)D-18 (Residual 2D+1D) network. This 3D CNN is highly effective for capturing spatio-temporal features through its use of factorized 3D convolutions (separate 2D spatial and 1D temporal operations). I use a Transfer Learning strategy, utilizing weights pre-trained on the Kinetics-400 dataset, a collection of 400 human action classes. A full fine-tuning approach was implemented, meaning the entire network was kept unfrozen to update its weights and fully adapt to the specific basketball action dataset.

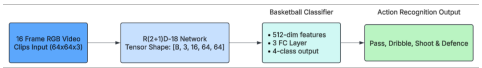

Figure 2: Low Level Diagram of Final Model Architecture

The model receives as input 16-frame RGB video clips of size 64x64 pixels transferred as a tensor with shape: [Batch = B, Channels = 3, Depth = 16, Height = 64, Width = 64] to classify the 4 basketball actions from the dataset. Then, I implemented my own final classification neural network instead of using the pre-trained model classification head. The structure is a three-layer fully connected network receiving the final 512-dimensional embedding vector of the R(2+1)D network. This classification head will produce, using ReLU activation function, this sequence: 1024->512->4 outputs, which represent the four basketball classes. The training methodology involves using cross-entropy loss, Adam optimizer with a learning rate of 1e-4, a batch size of 8 and 20 training epochs.

6. Baseline Model

For my baseline model, I implemented a three-dimensional Convolutional Neural Network (3D CNN) to classify the basketball action video clips. The benefit of 3D CNN is that the model convolves over both the spatial (height x width) and temporal (frame sequence dimensions. This process allows to learn spatiotemporal features such as the ball trajectories or player movements across consecutive frames. Each sample consists of a 16-frame grayscale video clip of size 64x64 pixels transferred as a tensor with shape: [Batch = B, Channels = 1, Depth = 16, Height = 64, Width = 64] to classify the 4 basketball actions from the dataset.

The model architecture consists of three 3D convolutional blocks, increasing the number of channels as follows: 1->16->32->64. Each block includes a 3x3x3 kernel, batch normalization, ReLU activation and 2x2 max pooling. After the 3D convolutions, the resulting feature map is flattened and passed through a fully connected layer to produce a 256-dimensional feature vector. Then, this is mapped to 4 output neurons, one for each class. The training methodology involves using cross-entropy loss as the loss function, a dropout rate of 0.3, Adam optimizer with a learning rate of 1×10^{-3}, batch size of 16 and 10 training epochs. It was able to achieve a training validation accuracy of 67%.

7. Quantitative Results

Regarding the quantitative results, the best validation accuracy achieved by my final model was 86.46% at epoch 16 during the training (Figure 3), with a corresponding training accuracy of 99.03%. This performance is a significant improvement of approximately 19% over the baseline 3d CNN's validation accuracy. This demonstrates that the model effectively learned spatiotemporal patterns, as the training curve shows gradual and smooth improvement. While the validation curve exhibits minor fluctuations, suggesting slight overfitting, the model's performance remains robust and acceptable. Also, I implemented a learning rate adjustment around every 10 epochs, which did help to stabilize the validation loss and improve the performance, showing effective tuning.

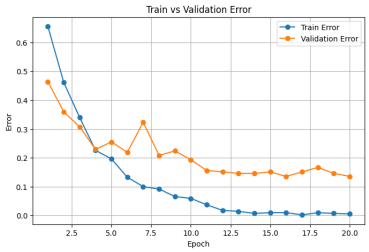

Figure 3: Training and Validation Error of the Final Model

8. Qualitative Results

Regarding the qualitative results, I collected information about the model using a confusion matrix on the validation data. The model performed consistently well across all four classes, with all metrics being robust (Recall $\geq 80\%$, F1-Score $\geq 82\%$, and Precision $\geq 83\%$) (see Appendix - Figure 6). The high recall means that the model can correctly identify all relevant instances of the class, and the high F1-score means the model performs consistently well across all four classes. There are still some areas of improvement, such as the pass action, which has the lowest recall of 80% likely due to the passing motions often resembling the initial phase of dribbling, making feature separation difficult. Also, this model receives RGB inputs instead of grayscale, as I found out that the model trained on RGB achieved a higher accuracy and better action separation due to the colour information to distinguish elements such as the ball or the players.

9. Evaluate Model on Test Data

Then, to evaluate the model on new data, 10% of my dataset for the 4 basketball teams was reserved for testing, serving as samples that were not used in any way to influence the tuning of hyperparameters. The model achieved a test accuracy of 81.96%. I evaluated the model on this new data using a confusion matrix. From this analysis, we see that the model performs generally well in identifying

the correct class. However, from the confusion matrix (Figure 4), we can also see that the model struggled to identify the pass movement, likely because this movement is very similar to the other classes.

I also introduced two new video clips from my own basketball games. One clip showed me shooting the basketball, and the other showed me dribbling. After processing the clips to match the input size used during training and passing them through the model, I obtained the following results: the shooting clip was classified as shoot (96.77% confidence) and the dribbling clip was classified as dribbling (99.74% confidence). These results show that the model meets expectations and is able to operate in various conditions to classify basketball actions accurately.

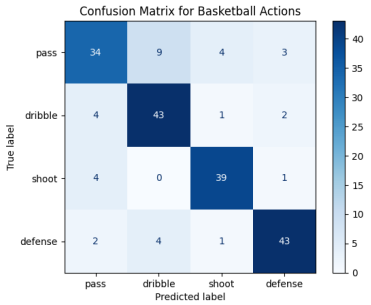

Figure 4: Confusion Matrix of the Final Model on the Test Data

10. Discussion

The results of my project show that my model performs well given the task of recognizing and classifying basketball actions such as shooting, dribbling, passing, and defence. The model achieved an accuracy of 81.96%, which is very good considering that I am gathering spatio-temporal feature information. These features are complex because, as mentioned before, some actions are very similar while others happen within a short time span. The results presented earlier show that the model was still able to learn from this and classify the actions correctly. It was interesting to see the power of 3D convolutions to learn spatio-temporal features across video frames and extract meaningful movement information, and to see that it works well in practice.

Through this project, I have learned different deep learning applications for classification, such as image and video classification. I was able to improve my knowledge of convolutions and processing data to work through a video dataset. I also learned how to use pre-trained models and understand their benefits in deep learning applications compared to building from scratch with less data.

11. Ethical Considerations

In relation to the training data, the Space Jam dataset may not reflect the real-world diversity of basketball players, including differences in size, skin colour, playing style, and camera angles. This could introduce bias and limit generalization if applied beyond controlled settings. Regarding the model, in professional contexts, if coaches or analytics teams use the system to assist player performance, it should be used responsibly, as misclassifications could negatively impact the coaching or training decisions if the system is relied on too heavily.

12. Project Difficulty / Quality

My project was difficult because it required using a 3D CNN, working with a video-based dataset instead of simple images, and handling long training times since each video had to be split into 16 frames. Despite all these challenges and the many attempts I made using 2D CNNs, 3D CNNs, and transfer learning, the final model still performed well and was able to meet the expectations for classifying the four basketball actions under many conditions. I also learned beyond the course material by researching 3D CNNs and their applications specifically for this project.

References

[1] J. Wang, L. Zuo, and C. Cordente Martínez, "Basketball technique action recognition using 3D convolutional neural networks," *Scientific Reports*, vol. 14, no. 1, Art. no. 13156, Jun. 2024. [Online]. Available: https://www.nature.com/articles/s41598-024-63621-8.

[2] J. Fan, C. Ma, J. Yao and T. Zhang, "NPU RGBD dataset and a feature-enhanced LSTM-DGCN method for action recognition of basketball players," *Applied Sciences*, vol. 11, no. 10, p. 4426, May 2021. [Online]. Available: https://www.mdpi.com/2076-3417/11/10/4426.

[3] B. Gan, F. Li, Q. Liu, S. Wang and S. Zhao, "BRGR-Net: An efficient basketball referee gesture recognition network," *Computers in Industry*, vol. 132, p. 103515, Jul. 2025. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0263224125024571.

[4] L. Ding, W. Tian and J. Xiao, "Basketball action recognition method of deep neural network based on dynamic residual attention mechanism," *Information*, vol. 14, no. 1, p. 13, Jan. 2023. [Online]. Available: https://www.mdpi.com/2078-2489/14/1/13.

[5] X. Meng, H. Shi and W. Shang, "Analysis of basketball technical movements based on deep learning," *Comput Intell Neurosci.*, vol. 2022, Article ID 9023221, Dec. 2022. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9023221/.

[6] S. Francia, "Classificazione di Azioni Cestistiche mediante Tecniche di Deep Learning", 2018. [Online]. Available: https://www.researchgate.net/publication/330534530_Classificazione_di_Azioni_Cestistiche_mediante_Tecniche_di_Deep_Learning.

[7] S. Francia, "SpaceJam: a Dataset for Basketball Action Recognition," GitHub repository, 2018. [Online]. Available: https://github.com/simonefrancia/SpaceJam. [Accessed: Sep. 26, 2025].

Appendix

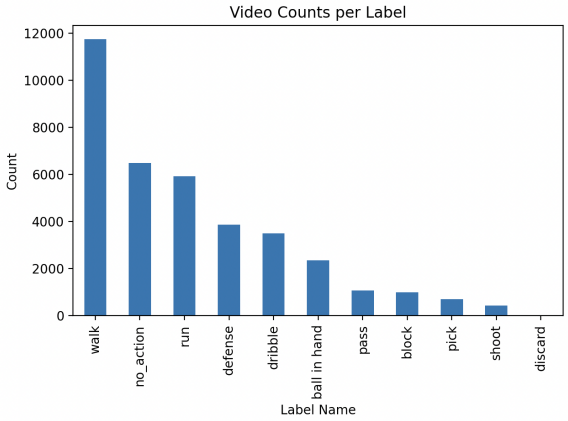

Figure 5: Class distribution and number of videos per category

Category:	Precision	Recall	F1-Score
Pass	0.83	0.80	0.82
Dribble	0.83	0.86	0.84
Shoot	0.95	0.95	0.95
Defense	0.86	0.86	0.86

Figure 6: Confusion Metrics of the Final Model on Validation Data